ROSE SWE Software Engineering

ROSE SWE

🔗 http://rose-swe.bplaced.net

🔗 http://www.cfg2html.com

📧 rose_swe@gmx.net

Enduser Program Documentation

This document provides comprehensive guidance for ROSE SWE software, documentation, and services. It details intended use, system configuration, and operational behavior, serving as an essential resource for administrators and end-users managing these solutions in production environments.

The information in this document is provided “as is” and is subject to change without notice. The information in this document, including products and software versions, is current as of the ‘Release Date’. ROSE SWE assumes no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein or the usage of the program itself.

Last Updated: 09-Nov-2026

Table of Contents

verinc - Version Increment Tool 3
1. 🚀 Features 3
2. 🏷️ Versioning Philosophy & Semantic Support 3
   2.1. Semantic Versioning Reference 3
   2.2. Overflow Behaviour 3
3. 📂 Supported File Types 3
   3.1. Common Use Cases: 3
4. 📖 Usage 4
   4.1. Options 4
   4.2. Notes 4
5. 🚦 Exit Codes 4
6. ✨ Examples 5

Table of Contents

verinc - Version Increment Tool

(c) by Ralph Roth, ROSE SWE

`verinc` is a robust, lightweight command-line utility written in Go (v1.25++) designed to automate the management of version strings within source code. It targets specific lines containing version tokens and applies custom incrementation rules, making it ideal for build pipelines and release management.

> 💡 **NOTE**
> We use this tool in-house for various projects e.g. for cfg2html or getrpm (both available on github too).

---

1. 🚀 Features

- **Smart Versioning**: Automatically handles `MAJOR.MINOR.PATCH` increments.
- **Token-Based Selection**: Processes only lines containing specific markers like `_VERSION`, `ProductVersion`, or `FileVersion`.
- **Overflow Logic**: Implements a custom rule where reaching a PATCH of 10 resets it to 1 and increments the MINOR version. Can be disabled with `--nowrap`.
- **Safe Updates**: Files are written atomically. Line endings (LF/CRLF) are preserved, file permissions are kept, and the original file remains untouched if anything goes wrong mid-write.
- **Multi-File Support**: Update multiple source files in a single command.
- **Detailed Feedback**: Verbose mode provides a clear "old -> new" transition log for every change.
- **Standard Library**: Built using only the Go standard library for maximum portability.

---

2. 🏷️ Versioning Philosophy & Semantic Support

While `verinc` follows a custom overflow rule (e.g., `3.19.9` ⟶ `3.20.1`), it is fundamentally designed to support the structure of **Semantic Versioning (SemVer)**.

2.1. Semantic Versioning Reference

The tool operates on the standard `MAJOR.MINOR.PATCH` format:

- **MAJOR**: Incremented for incompatible changes. Use `-j` or `--major`.
- **MINOR**: Incremented for added functionality. Use `-m` or `--minor`.
- **PATCH**: Incremented for backwards-compatible bug fixes. This is the **default** behavior.

2.2. Overflow Behaviour

By default, when PATCH would reach 10, `verinc` wraps: `3.20.9` ⟶ `3.21.1`. If you prefer a plain increment without wrapping (`3.20.9` ⟶ `3.20.10`, or `1.7.9` ⟶ `1.7.10`), pass `-n` or `--nowrap`. The `-m` / `-j` bumps are unaffected by `--nowrap` and still explicitly reset PATCH to 1.

---

3. 📂 Supported File Types

`verinc` is language-agnostic and processes files based on line content rather than extension.

3.1. Common Use Cases:

- **Source Code**:

© 1989-2026 by ROSE SWE, Ralph Roth - All Rights Reserved

  - **Pascal/Delphi/Lazarus**: Updates constants or comments marked with `_VERSION`.
  - **Go/C/C++**: Updates string constants or header macros.
- **Windows Resource Files (`.rc`)**: Targets `FILEVERSION` and `PRODUCTVERSION` definitions.
- **Compiler & Build Metadata**:
  - **JSON Configs**: Updates `ProductVersion`, `Version`, or `FileVersion` keys found in `package.json` or manifests.
  - **Project Files**: Updates XML or plain-text metadata used by compilers.

---

4. 📖 Usage

```
verinc [options] file1 [file2 ...]
```

4.1. Options

Short	Long	Description
`-?`, `-h`	`--help`	Show detailed help and documentation.
`-V`	`--version`	Show current version of the `verinc` tool.
`-v`	`--verbose`	Enable verbose output (shows specific line changes).
`-g`	`--get`	Return the first matching version string to stdout (useful for Makefiles/scripts).
`-m`	`--minor`	Bump **MINOR** version and reset **PATCH** to 1.
`-j`	`--major`	Bump **MAJOR** version and reset **MINOR/PATCH** to 0/1.
`-n`	`--nowrap`	Bump **PATCH** without wrapping (`1.7.9` ⟶ `1.7.10`).

4.2. Notes

- Only lines containing a recognised version token (`_VERSION`, `ProductVersion`, `FileVersion`, or `"Version"`) are touched.
- The first `MAJOR.MINOR.PATCH` found on such a line is updated.
- Files are updated atomically: line endings and permissions are preserved, and an interrupted run cannot corrupt your source.
- `-h` and `--help` print the help screen and exit with code `0`.

---

5. 🚦 Exit Codes

`verinc` returns specific exit codes for easy integration with CI/CD scripts:

- `0`: Success ✅
- `1`: Generic error
- `2`: Invalid command line arguments
- `3`: No files provided
- `4`: File open/read error
- `5`: No version tokens (`_VERSION`, etc.) found in any file
- `6`: Version string parse error
- `7`: File write error

© 1989-2026 by ROSE SWE, Ralph Roth - All Rights Reserved

6. ✨ Examples

Basic patch increment:

```
verinc main.pas
```

Minor version jump with verbose output:

```
verinc -m -v ~/src/project/version.go
```

Major version jump:

```
verinc -j constants.pas
```

Patch increment without overflow wrap (1.7.9 → 1.7.10):

```
verinc -n main.pas
```

Updating multiple files at once:

```
verinc -v config.json constants.pas README.md
```

Get first matching version string (for Makefiles/Bash scripts):

```
verinc --get version.go
# Output: 2.05

# Useful in Makefiles or shell scripts:
VER=$(verinc -g main.pas)
echo "Version is $VER"
```

© 1989-2026 by ROSE SWE, Ralph Roth - All Rights Reserved